

```
In [...] import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
from matplotlib.animation import FuncAnimation, PillowImage
```

Histograms

By partitioning the data into bins (intervals) and showing the frequency of data points that fall within each bin, a histogram shows how a numerical data is distributed. They are useful in data analysis to identify patterns or anomalies in the distribution.

From Wikipedia

"A histogram is a visual representation of the distribution of quantitative data. The term was first introduced by Karl Pearson. To construct a histogram, the first step is to "bin" (or "bucket") the range of values— divide the entire range of values into a series of intervals—and then count how many values fall into each interval. The bins are usually specified as consecutive, non-overlapping intervals of a variable. The bins (intervals) are adjacent and are typically (but not required to be) of equal size."

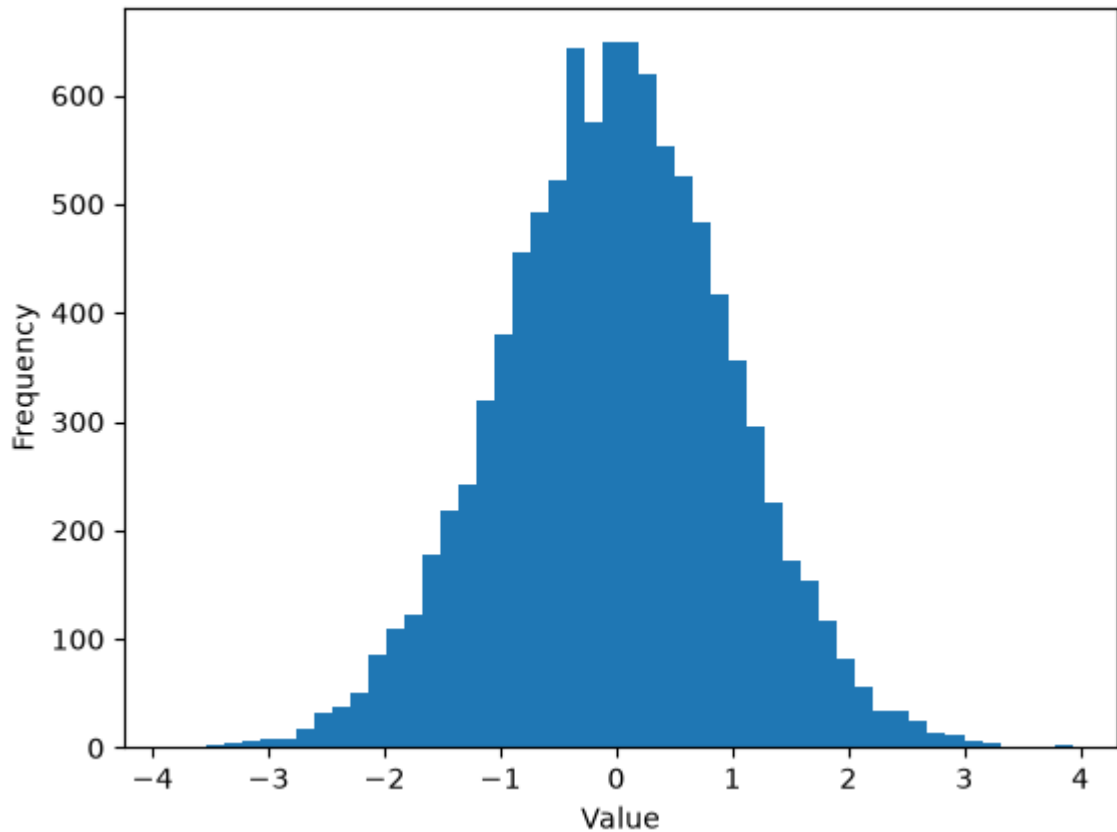
"Histograms give a rough sense of the density of the underlying distribution of the data, and often for density estimation: estimating the probability density function of the underlying variable."

```
In [...] # Creating Histograms with Matplotlib

# random data
data = np.random.randn(10000)
data
data.std()

# histogram
plt.hist(data, bins=50)
plt.xlabel("Value")
```

```
plt.ylabel("Frequency");
```

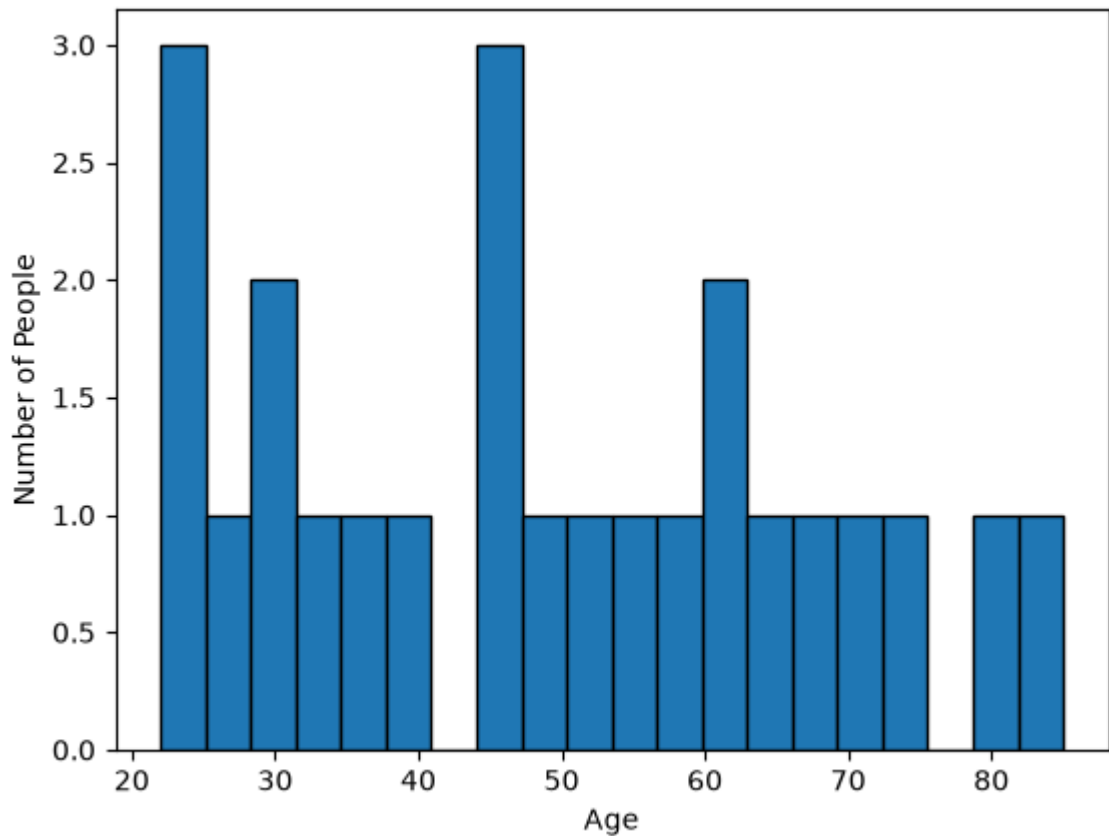


```
In [ ... # Another example: ages of people in a survey
ages = np.array([22, 23, 25, 27, 29, 30, 32, 35, 40,

len(ages)
print(ages.min(), ages.max())

plt.hist(ages, bins=20, edgecolor="black")
plt.xlabel("Age")
plt.ylabel("Number of People")
pass;
```

22 85



In [... *# check seaborn too, it's very useful for data analys*

Playing with random numbers

```
In [... import random

# random.choice(["A", 1, 1.2])

def coin_toss():
    return random.choice([1, 0])

# coin_toss()
# # Simulate N coin tosses
# for _ in range(10):
#     coin_toss()

N = 1000 # increase the number of tosses. what do yo
tosses = [coin_toss() for _ in range(N)] # list comp
# print(tosses)
count_H = 0
count_T = 0
for toss in tosses:
    if toss == 1:
```

```

        count_H = count_H + 1
    else:
        count_T = count_T + 1
print(count_H, count_T, count_H / len(tosses), count_
plt.hist(tosses)#, bins=[-0.5, 0.5, 1.5], density=True
plt.xticks([0, 1])
# plt.grid();

```

488 512 0.488 0.512

```

Out[... (array([512.,  0.,  0.,  0.,  0.,  0.,  0.,
 0.,  0., 488.]),
        array([0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8,
 0.9, 1. ]),
        <BarContainer object of 10 artists>)
Out[... ([<matplotlib.axis.XTick at 0x168bb34d0>,
<matplotlib.axis.XTick at 0x168b5fc20>],
[Text(0, 0, '0'), Text(1, 0, '1')])

```



```

In [... [i for i in range(3)]

```

```

Out[... [0, 1, 2]

```

```

In [... # Counting heads in two coin tosses. label 'H' by 1 a
        trials = 1000
        for i in range(1, trials + 1):
            # print(f"Number of trials = {i}")

```

```

count = 0
sum = []
for j in range(1, i + 1):
    # draw randomly from {0,1}, TWICE
    random_array = np.random.binomial(1, 0.5, size=2)
    sum_i = np.sum(random_array)
    # print(f"Outcome {j} = {random_array} and number of ones is {sum_i}")
    sum.append(sum_i)
# print(sum)
# Counter([1,2,2,3,4])
print(Counter(sum))
plt.bar(Counter(sum).keys(), Counter(sum).values(), width=0.2)
# pass;

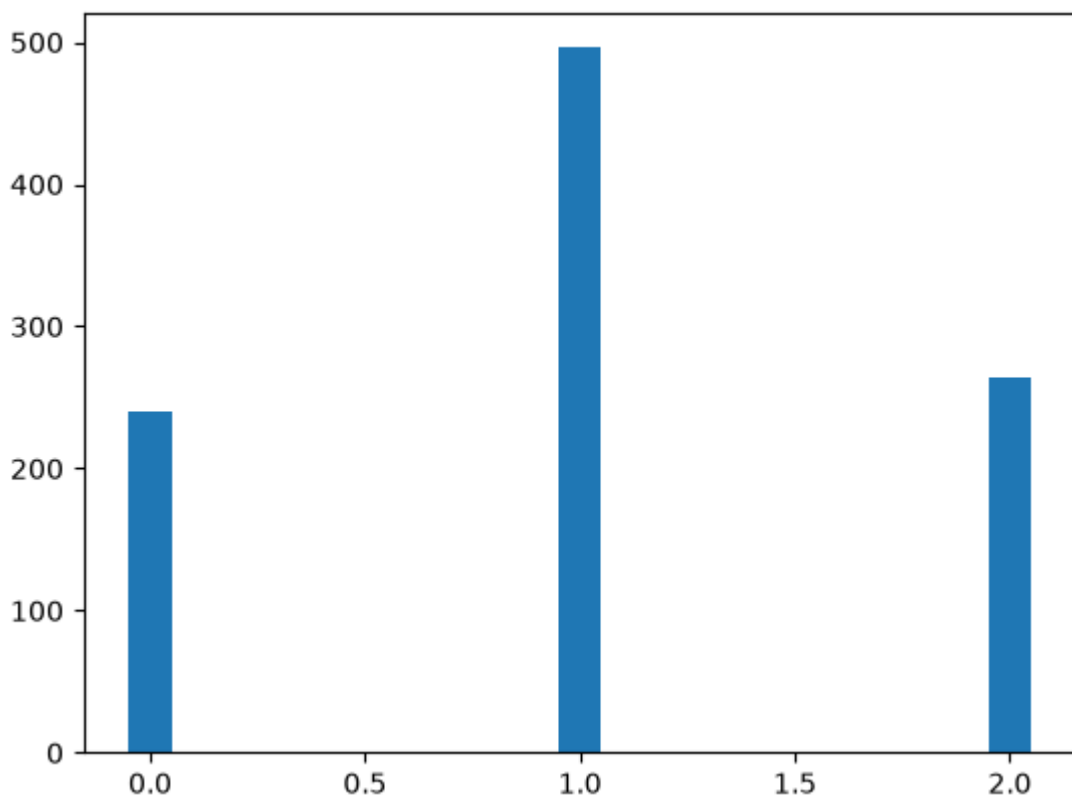
```

```

Counter({np.int64(1): 496, np.int64(2): 264, np.int64(0): 240})

```

Out[... <BarContainer object of 3 artists>



```
In [... print(np.random.choice(["H", "T"]))
```

T

```
In [... n_steps = 1000
n_walks = 100

# generate random steps of -1 or 1 for each walk

```

```

# documentation: https://numpy.org/doc/stable/referen
# IMPORTANT: Always look up documentation! Other than

steps = np.random.choice([-1, 1], size=(n_walks, n_st
# print(f"Step chosen: {steps}")
# compute cumulative sum along each row to get final
walks = np.cumsum(steps, axis=1)
# print(f"Position at each time step: {walks}")

for i in range(n_walks):
    plt.plot(walks[i], label=f"Walk {i+1}")

plt.title("Multiple 1D Random Walk Trajectories")
plt.xlabel("Steps")
plt.ylabel("Position")
pass;
# plt.legend();

```



```

In [... # Number of steps and number of walks
n_steps = 100
n_walks = 100000

steps = np.random.choice([-1, 1], size=(n_walks, n_st
walks = np.cumsum(steps, axis=1)

```

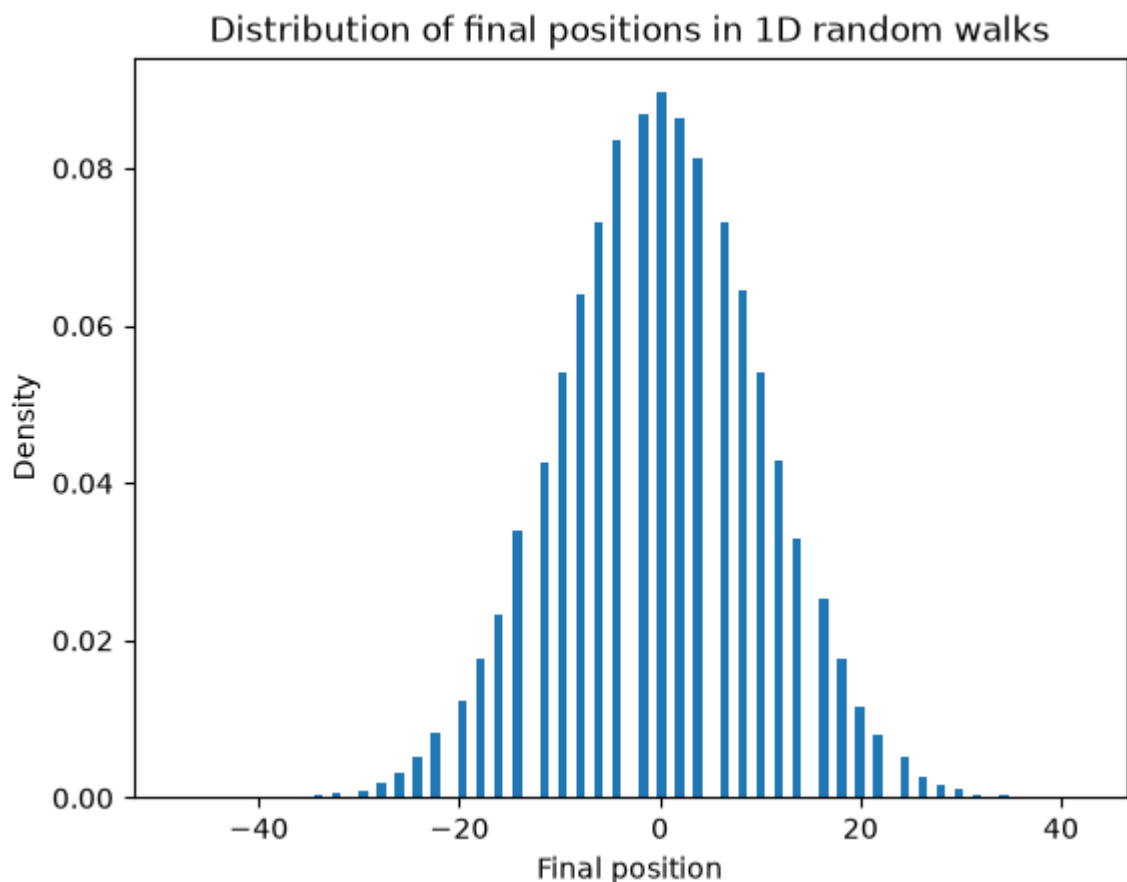
```

# print(walks)
# as I mentioned, the last element is the final position
final_positions = walks[:, -1]
# print(final_positions)

# Plot the distribution of final positions. Remember:

plt.hist(final_positions, bins=100, density=True)
plt.title("Distribution of final positions in 1D random walk")
plt.xlabel("Final position")
plt.ylabel("Density")
pass;

```



Linear regression: minimal example

```

In [... # install scikit learn for this. https://www.youtube.com/watch?v=...

from sklearn.linear_model import LinearRegression

X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1) # makes
# print(X)
y = np.array([3, 6, 2, 9, 11])

```

```

model = LinearRegression()
model.fit(X, y)

slope = model.coef_[0]
intercept = model.intercept_

print(f"Slope ($m$) = {slope:.2f}") # print first two
print(f"Intercept ($c$) = {intercept:.2f}")

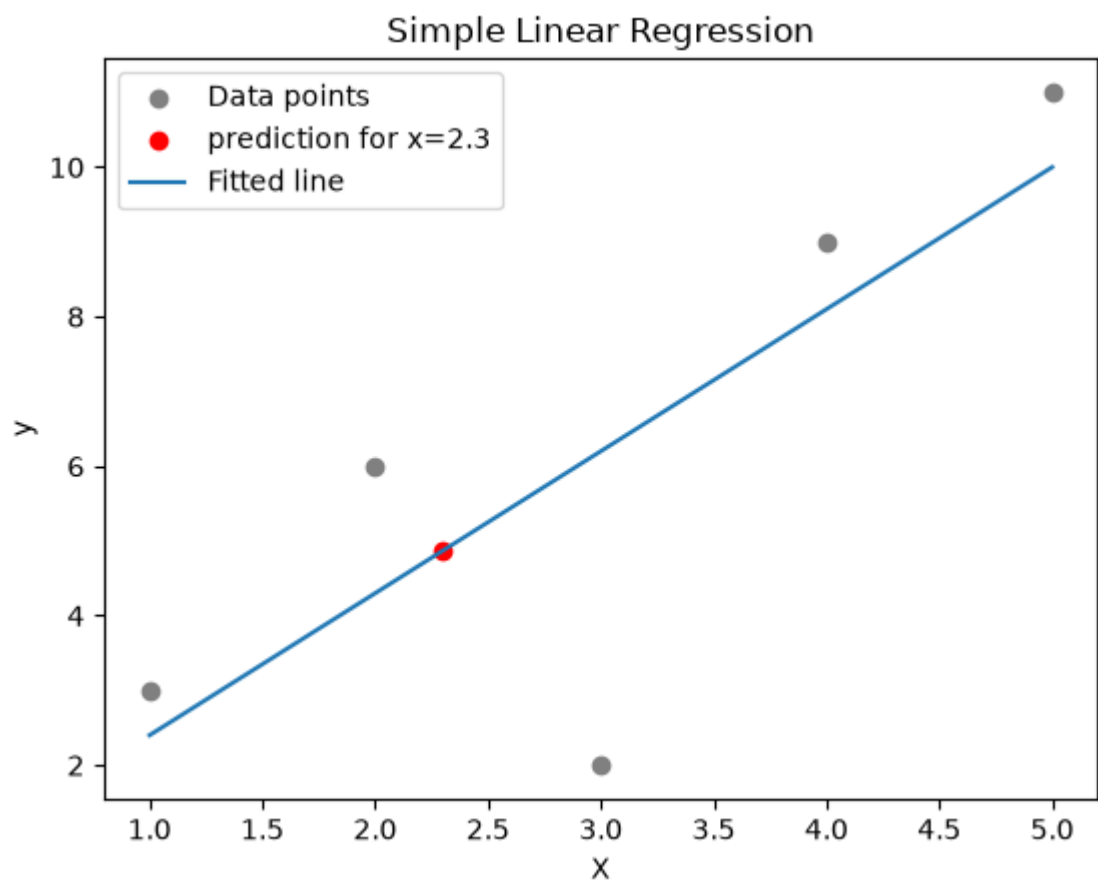
# check your fitted line for a `new` point. this is w
x_test = 2.3
X_new = np.array([[x_test]])
y_pred = model.predict(X_new)

print(f"y(x={x_test}) = {y_pred[0]:.2f}")

plt.scatter(X, y, label="Data points",color='gray')
plt.scatter(x_test,y_pred[0],color='red',label=f'pred
plt.plot(X, model.predict(X), label="Fitted line")
plt.xlabel("X")
plt.ylabel("y")
plt.title("Simple Linear Regression")
plt.legend()
pass;

```

Slope (\$m\$) = 1.90
 Intercept (\$c\$) = 0.50
 y(x=2.3) = 4.87



In [...]